PROJECT REPORT

PROJECT TITLE : HOUSE PRICE PREDICTION AND RATE ANALYSIS

DATE: 28-03-2026

SUMMITTED BY: JEEYA

REGISTATION NO: 25BAI10808

SUMMITTED TO: SUNEET JOSHI

COURSE: AIML

UNIVERSITY: VIT BHOPAL UNIVERSITY

1. Introduction

The House Price Prediction & Rate Analysis project is a data science application designed to estimate residential real estate values. By leveraging historical data from California districts, the system identifies patterns between physical attributes (like room count or age) and geographic locations to provide objective price estimates.

2. Problem statement

Real estate prices vary widely depending on income levels, number of rooms, population density, and location. Manually estimating a fair price for a house is difficult and often inaccurate. The goal of this project is to build a regression model that can automatically predict the median house value of a district, given a set of numeric features such as median income, average number of rooms.

3. Functional requirements

- Load the California housing dataset programmatically.scikit-learn
- Split the data into training and testing sets.
- Scale the input features using standardization.
- Train a Linear Regression model to predict house prices.
- Compute evaluation metrics: RMSE and R^2 score.
- Generate visualizations:
    - Scatter plot of actual vs predicted prices.
    - Histogram showing the distribution of house prices.
- Print basic statistics about price (min, max, mean, median, percentiles).

4. Non-functional requirements

- The model should train and evaluate within a few seconds on a normal laptop.
- Code should be simple, readable, and modular enough for beginners to understand.
- Visualizations should clearly show how well the model fits the data.
- The solution should be reproducible: same results when re-run with the same random seed.

5. System architecture

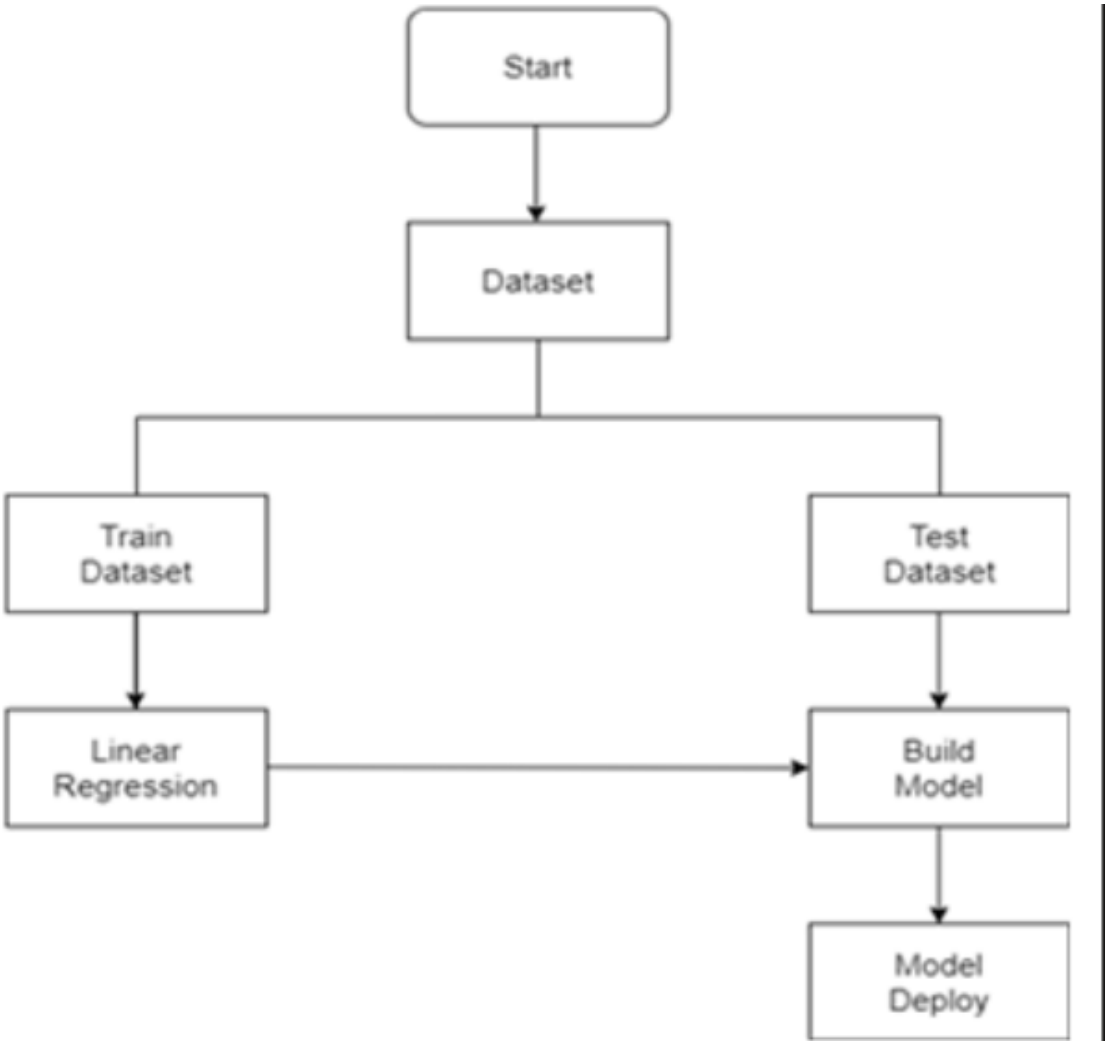
The system follows a simple pipeline architecture:

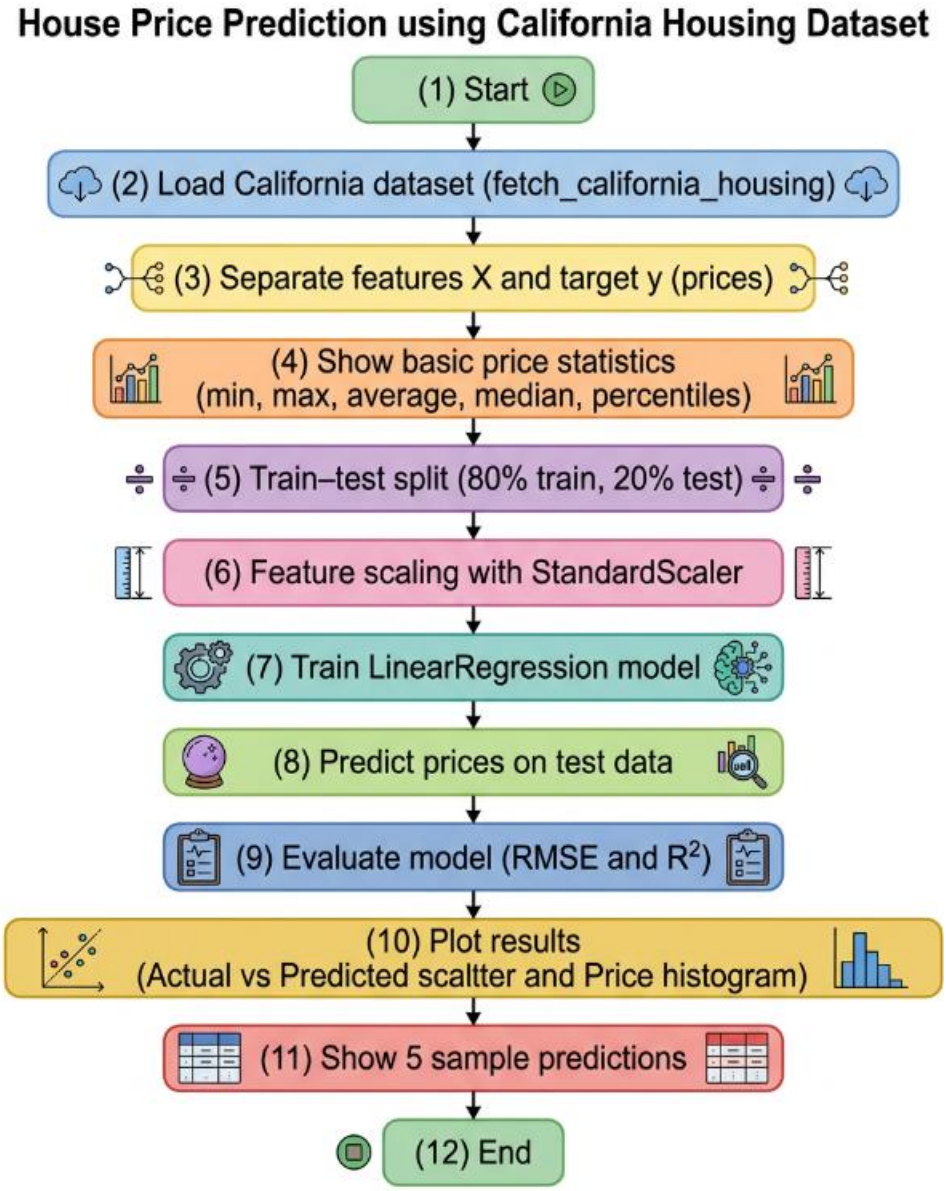
1. **Data Source Layer**
    - Uses fetch_california_housing() from scikit-learn to download and load the dataset.scikit-learn
2. **Preprocessing Layer**
    - Splits the dataset into training and test sets using train_test_split.
    - Applies StandardScaler to normalize feature values.
3. **Model Layer**
    - Trains a LinearRegression model on the scaled training data.
4. **Evaluation Layer**

- Uses mean_squared_error and r2_score to calculate RMSE and R^2.

5. **Visualization & Reporting Layer**
    - Uses Matplotlib to plot predictions and price distributions.
    - Prints numeric summaries and sample predictions to the console.

6. **Design diagram**





7. Design decisions and rationale

Algorithm (Linear Regression):

Chosen for its interpretability. It allows us to see the direct relationship between features and price.

Scaling (StandardScaler):

Since features like "Average Rooms" and "Population" have different units, scaling them to a mean of 0 and variance of 1 prevents larger numbers from dominating the model.

Split Ratio (80/20):

A standard Pareto-based split to ensure enough data for learning while maintaining a robust test set.

8. Implementation details

- Language: Python
- Libraries: numpy, matplotlib, scikit-learn (datasets, model_selection, preprocessing, linear_model, metrics)
- Steps in your code:
    - Load dataset and separate X (features) and y (target).
    - Print min, max, average, median, 25th and 75th percentiles of y.
    - Split into 80% training and 20% test set.
    - Fit StandardScaler on X_train and transform X_train, X_test.
    - Train LinearRegression on scaled training data.
    - Predict on scaled test data.
    - Compute RMSE and R^2, and print them in a formatted string.
    - Plot:
        - Scatter: y_test vs y_pred with a red reference line (perfect prediction).
        - Histogram: distribution of all house prices in dollars.
    - Randomly choose 5 test samples and print actual price, predicted price, and absolute error.

9. Screenshot

INPUT :

```python
# House Price Prediction + Rate Analysis
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load data
housing = fetch_california_housing()
X =
(variable) housing: tuple[Bunch, tuple]
y = housing.target  # Prices in $100,000 units

print("📊 House Price Statistics:")
print("Min price:  $", np.min(y)*100000, "(Cheapest)")
print("Max price:  $", np.max(y)*100000, "(Most expensive)")
print("Avg price:  $", np.mean(y)*100000, "(Average)")
print("Median:     $", np.median(y)*100000)
print("25th%:      $", np.percentile(y, 25)*100000, "(Lower middle)")
print("75th%:      $", np.percentile(y, 75)*100000, "(Upper middle)")

# Your existing pipeline (short version)
```

```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Predictions & evaluation
y_pred = model.predict(X_test_scaled)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"\n✅ Model: RMSE=${rmse*100000:.0f} | R²={r2:.2f}")

# Plot
plt.figure(figsize=(10,4))
plt.subplot(1,2,1)
plt.scatter(y_test, y_pred, alpha=0.5)
plt.plot([0,5],[0,5],'r--')
plt.xlabel("Actual ($100k)"); plt.ylabel("Predicted ($100k)")
plt.title("Predictions")

plt.subplot(1,2,2)
```

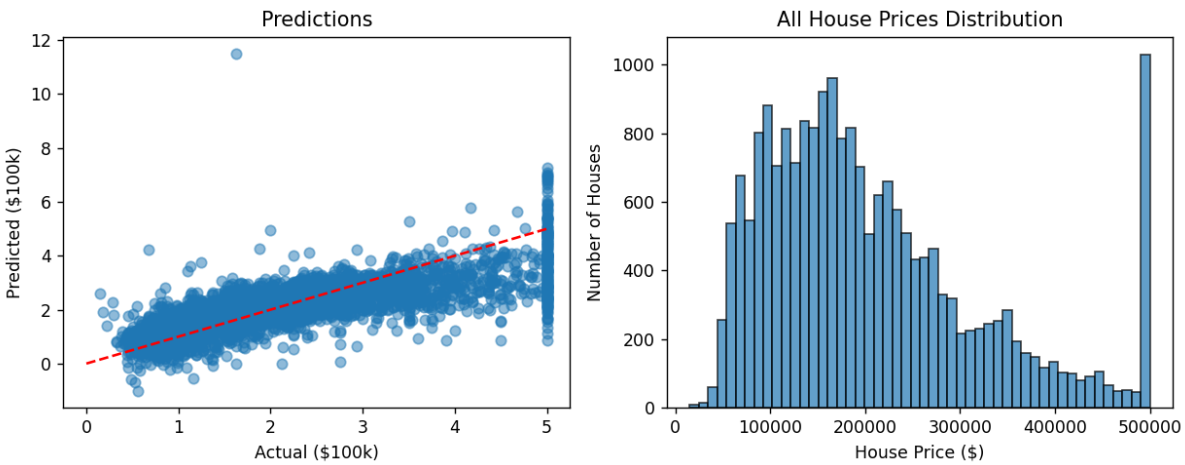
```python
plt.subplot(1,2,2)
plt.hist(y*100000, bins=50, alpha=0.7, edgecolor='black')
plt.xlabel("House Price ($)")
plt.ylabel("Number of Houses")
plt.title("All House Prices Distribution")
plt.tight_layout()
plt.show()

# 5 Example predictions
print("\n🏠 5 Sample Predictions:")
sample_indices = np.random.choice(len(X_test), 5, replace=False)
for i, idx in enumerate(sample_indices):
    actual = y_test[idx] * 100000
    pred = y_pred[idx] * 100000
    error = abs(actual - pred)
    print(f"House {i+1}: Actual=${actual:.0f} | Pred=${pred:.0f} | Error=${error:.0f}")
```

OUTPUT:

```
📊 House Price Statistics:
Min price:  $ 14999.000000000002 (Cheapest)
Max price:  $ 500000.99999999994 (Most expensive)
Avg price:  $ 206855.81690891474 (Average)
Median:     $ 179700.0
25th%:      $ 119600.0 (Lower middle)
75th%:      $ 264724.99999999994 (Upper middle)

✅ Model: RMSE=$74558 | R²=0.58
```



10. Testing approach

- **Unit-level checks**:

    - Verified that data loading works and shapes of X and y are correct.
    - Checked that scaling does not change the number of samples or features.
- **Model evaluation**:
    - Calculated RMSE on test data to estimate the average prediction error in USD.
    - Calculated R^2 to understand the proportion of variance explained by the model.
- **Sanity checks**:
    - Plotted predicted vs actual values to visually confirm that many points lie near the diagonal line.
    - Checked randomly selected test samples to see if predictions are within a reasonable range of actual prices.

11. Challenges faced

- Understanding the meaning of the target variable MedHouseVal in units of 100,000 and converting it into real-dollar prices.
- Choosing appropriate evaluation metrics and interpreting RMSE and R^2 in a meaningful way.
- Getting the correct preprocessing order: split → scale → train, to avoid data leakage from the test set.
- Installing and configuring Python libraries (scikit-learn, matplotlib) and dealing with pip-related issues.

12. Learning and key takeaways

- Learned how to use a real-world dataset and perform end-to-end regression analysis in Python.
- Understood the importance of train–test split to evaluate how the model generalizes to unseen
- Saw how feature scaling can improve model training stability and performance.
- Gained practical experience with core metrics (RMSE and R^2) and how they reflect model accuracy in real units (dollars).

- Learned to use plots to quickly check model quality and detect underfitting or noise.

13. Future enhancement

- Try more advanced models such as Ridge, Lasso, Random Forest, or Gradient Boosting to improve accuracy.
- Perform feature engineering, such as combining or transforming existing features (e.g., rooms per person, distance to coast).
- Build an interactive web app using Streamlit where users can input custom feature values and get live price predictions.
- Add cross-validation to get a more robust estimate of model performance across different splits.
- Integrate geospatial visualizations (maps) to show how prices change across California districts.

14. References

- California Housing Prices dataset description (Kaggle)
- Regression models for California housing price prediction (tutorials)
- Required libraries installed via pip (numpy, matplotlib, scikit-learn)